PRODUCT + ENGINEERING FIELD GUIDE

Startup.life

The game, its technology, and what is inside it

A practical inventory of the current codebase: the island world, player systems, learning tools, backend, APIs, integrations, hosting, and honest implementation limits.

Edition Current repository snapshot | 19 October 2026
Project Startup.life / Dev District
Primary experience Browser game and installable PWA

01 | What the game is

Startup.life combines a third-person island life-simulation with developer learning, careers, and social play.

Players explore a stylized Lagos-inspired island, choose a development career, visit buildings, complete coding challenges, and use an in-world phone and laptop. Guests can explore; account-backed features require signing in. The web client renders the world in the browser, while the Node server handles accounts, saved game data, and real-time play.

The codebase contains two related client tracks

Track	What it contains	Status
Browser game	React + TypeScript interface, Three.js island, Express API, Socket.IO multiplayer, PWA installation.	Playable web experience; current runtime documented by the app and server.
GDevelop project	A separate GDevelop project with the Lagos Shadow Frontier GLB, a third-person avatar/camera, desktop movement, and touch controls.	Starter / migration track; it is not the same live runtime as the React web app. Online services need an integration bridge.

Product and presentation

- Startup.life welcome page and original logo mark, with a short build-stack slideshow.
- Mobile-friendly browser interface, desktop controls, phone and laptop overlays, and an installable PWA shell.
- Anime-inspired cel-shaded 3D presentation built from original procedural scene geometry and game code.
- The product is independent. Learn2Earn and Talent Nation links are references; the game does not claim to be their official service.

Project boundary: This repository is an original game prototype inspired by broad references. It is not the proprietary source code for lagoslife.app or any other referenced game.

02 | Technology used to build it

The active web client and backend share TypeScript models and game content where practical.

Area	Technology	What it does
Browser UI	React, TypeScript, TSX, CSS	Screens, forms, player HUD, phone/laptop apps, and interface state.
Build and dev server	Vite, @vitejs/plugin-react	Local development, API/WebSocket proxy, optimized production assets.
3D world	Three.js, WebGL, OrbitControls	Procedural island, city, buildings, avatars, camera, interactions, effects.
Server runtime	Node.js, TypeScript, Express 5	HTTP routes, auth/session checks, game data, uploads, and serving built web assets.
Realtime multiplayer	Socket.IO + socket.io-client	Online presence, nearby movement, chat, and live Trust Protocol rounds.
Database currently used	SQLite via better-sqlite3	Local runtime store for account and game state.
Content definitions	JSON + TypeScript	Places, homes, jobs, coding quests, lessons, items, terminal reference.
Mobile installation	Web app manifest + service worker	Add-to-home-screen PWA, app shell and static asset caching.
Deployment	Render web service + GitHub source	Builds the web client and starts the Express/Socket.IO server.

Main source folders

Path	Contents
frontend/src/	React application, ThreeCity renderer, learning hub, terminal, GitHub UI, device apps, CSS.
frontend/public/	PWA manifest/service worker/icons and source GLB asset.
backend/server/	Express routes, auth, SQLite access, OAuth, file operations, admin routes, Socket.IO.
backend/data/	Programming-resource catalogue and local runtime/user uploads (excluded from Git where appropriate).
backend/supabase/	Supabase config and SQL migrations; schema work, not the active runtime database adapter.
shared/data/	Shared JSON definitions for world, careers, learning, inventory and places.
gdevelop/	Separate GDevelop starter and migration notes.

03 | World and gameplay

A walkable browser-rendered island with places, player interaction, and progression systems.

The Three.js scene is generated procedurally in `frontend/src/ThreeCity.tsx`. It creates terrain, coastal water, roads, city blocks, buildings, venues, map markers, a stadium, beach elements, player avatars, and effects. The scene has a third-person follow camera plus inspect/orbit modes. Taps/clicks use scene picking; movement is smoothed in the animation loop.

Places and activities

- Tech Place: central blue-glass developer campus with reception, coding labs, shared recreation floor, leadership floor, and Yabuka canteen reached by elevator/security flow.
- Legend House: residential/rest destination; The Commons: social gathering space; Tech Noodle: food stall/shop.
- Thirty-two shared venue/place records include developer studios, learning/culture locations, markets, fitness and wellness, transport, recreation, and the BuildFest Stadium.
- Named destinations open their information/activity flow. Other modeled buildings have a generic explore/meet/rest fallback; the experience is not a fully pathfindable interior simulation in every structure.
- Interior layouts are rendered as compact isometric/cutaway rooms, with floor switching and venue-specific room dressing where implemented.

Character, movement, and environment

- Generated stylized 3D avatar models and selectable player appearance; connected users appear as remote avatars. Ambient human NPC population is removed.
- Desktop: WASD/arrow movement, faster movement, camera modes, and pointer selection. Mobile: touch joystick/controls and lighter rendering settings.
- Island map and destinations are available through the Phone; keyboard, touch, and camera interactions are handled by the client.
- The supplied Lagos Shadow Frontier GLB is present for the separate GDevelop starter. The current Three.js map is procedurally generated and does not load that model.

Game loop and rewards

Players select one of 13 career/task types, practice with 16 coding quests and 26 learning units, complete work tasks, collect a small set of items, visit/buy homes, and earn fictional in-game XP/currency for supported first completions. Rift combat and Trust Protocol social deduction are additional game loops.

04 | Phone, laptop, and learning apps

In-world devices organize the support features so the island remains the primary play space.

Device/app	What is implemented
Phone: Island map / Places	Player location, districts, locations, navigation, and online-player context.
Phone: Profile / account	Sign-up, sign-in, player profile and progress access; session-backed features require the server.
Phone: Dispatch	Food/ride/roadside request demonstration. It is a simulation, not a live dispatch network.
Phone: Search / links	Opens public web search and developer resources in the browser.
Phone: inventory / messages	Collected items, community/chat entry, and profile shortcuts.
Phone/laptop: GameUp	Installed app view links to the separately hosted GameUp games hub; games are not bundled inside this game repository.
Laptop: quests / careers	Career paths, objectives, coding tasks, and progression context. Career data includes frontend, backend, full-stack, product design, security, game development, AI and industry-focused roles.
Laptop: Linux terminal	Safe in-game command emulator, reference guide, editable simulated project files and browser-local workspace. No shell commands execute on the device or server.
Laptop: Learn2Earn-inspired hub	Original learning content: overview, beginner drills, projects, book catalog, and video library. It is not an official Learn2Earn service.
Laptop: GitHub	Connect/disconnect flow, profile, repositories, organizations, and repository links when the OAuth app credentials and callback are configured.
Laptop: developer demos / files	Developer role can submit project demo videos and share files. Grower role can browse demos with repository links. Server applies file-size and access checks.

Learning content

The included curriculum spans programming logic, debugging, web requests, HTML/React, Python functions and API endpoints, Node APIs, relational data, Git collaboration, security, testing, cloud/deployment, Go, Spring, ASP.NET, and Laravel. Coding tasks use original fill-in/code reasoning prompts. Signed-in completions are checked and persisted by the backend; guest progress uses browser-local storage.

Three account roles: Beginner focuses on learning/social use; Developer gets project/demo publishing tools; Grower can review developer demos. Career paths can remain visible across roles, while access to role-specific opportunities is gated.

05 | Backend, APIs, and realtime systems

The Express service serves the application and acts as the trusted boundary for accounts and saved state.

The Node server handles sign-up/login/logout, cookie sessions, profiles, career selection, job completion, learning/coding progress, homes, inventory, chat, book/video library, GitHub OAuth, uploads, reports, admin publishing, and health checks. Passwords are hashed; session identifiers are stored as hashes and sent in protected cookies. GitHub access tokens are encrypted server-side when OAuth is configured.

First-party HTTP API groups

Route group	Purpose
/api/health; /api/places; /api/jobs; /api/items; /api/homes	Health and public game catalog data.
/api/auth/*; /api/me; /api/player	Account sessions and player profile.
/api/jobs/*; /api/learning/*; /api/coding/*	Career selection, tasks, learning and coding progress.
/api/inventory/*; /api/homes/*	Collectibles, owned items and home actions.
/api/chat/*; /api/leaderboard	Chat history/reporting and player ranking.
/api/learn/books/*; /api/learn/videos	Learning library catalog, purchase/download, and video listing.
/api/github/*	OAuth connect/callback, status, profile, repositories, organizations, profile edits, disconnect.
/api/demos/*; /api/files/*	Demo upload/playback and authorized player file sharing.
/api/admin/*; /api/internal/admin/*	Staff publishing/moderation and protected admin metadata endpoints.

Socket.IO events

Socket.IO authenticates the account session and sends player join/move/leave and world counts. The client uses spatial rooms so nearby players receive movement updates. Realtime event groups also handle chat, Trust Protocol round start/role/meeting/vote/results, and Rift entry/exit interactions.

Uploaded demo videos are limited to 50 MB and player file sharing to 15 MB. Access to downloads is checked against sender/recipient or authorized demo-view access. Admin book content accepts PDFs; administrators can curate YouTube HTTPS links.

06 | External services, data, and hosting

What the project calls or links to, and where live data is actually stored.

Service / API	Use in the game	Runtime status
GitHub OAuth + GitHub REST API	Consent, authenticated profile, repository list, organization list and profile updates.	Implemented server-side; requires valid GitHub OAuth environment settings.
YouTube privacy-enhanced embed	Admin-curated lesson links embedded in the video library.	Used for curated videos; not a video hosting service.
Google Fonts	DM Sans and Manrope typography in the web UI.	External font stylesheet; browser may fall back when offline.
GameUp hosted app	Phone/laptop link to the separate game hub.	External link to another Render-hosted service.
Google web search / public references	Phone search and shortcuts to GitHub, MDN, Stack Overflow, Learn2Earn/Talent Nation and other learning links.	Outbound links; not game-owned APIs.
Supabase	SQL config/migrations for a future server-owned database schema.	Not connected to active game runtime; live app currently reads/writes SQLite.
Render	Hosts the combined frontend + Node API/Socket.IO service.	Configured in render.yaml; free service filesystem is ephemeral.

Storage and deployment facts

- Current server database: local SQLite file at backend/data/lagos-life.sqlite (or DATABASE_PATH). It stores accounts and supported game state.
- Some guest preferences and learning/inventory progress are kept in browser local storage.
- Uploaded PDFs and player media are stored on the server filesystem unless PRIVATE_UPLOADS_PATH points at persistent storage. Render free filesystem may reset after restart or redeploy.
- Supabase project/schema work exists in repository and the hosted database is provisioned, but the application has not switched its runtime adapter or migrated existing SQLite data.
- PWA service worker caches public app-shell/static assets, not private account data. Login, multiplayer and saved actions need an internet connection.

Important: Do not treat the current Render + SQLite deployment as durable production storage. A real Supabase/Postgres adapter, migration, backups, and persistent object storage still need to be completed and verified.

07 | Repository map, build, and boundaries

A quick reference for developers continuing the project.

Location	What it contains
frontend/src/App.tsx	Main application state, controls, panels, API calls and Socket.IO client.
frontend/src/ThreeCity.tsx	Three.js city world, avatar rendering, interiors, cameras, effects and interactions.
frontend/src/LearnHub.tsx; GameTerminal.tsx; GitHubApp.tsx; DeviceApps.tsx	Learning, terminal, GitHub and demo/file/GameUp interfaces.
frontend/src/StartupWelcome.tsx + CSS	Welcome/intro screen and build slideshow.
backend/server/index.ts	Express API, auth, persistence, integrations, admin routes and Socket.IO server.
shared/data/*.json	Places (32), jobs (13), coding quests (16), learning units (26), homes (4), items (3), Linux command guide.
backend/supabase/migrations/	Database schema/migrations for the planned Supabase runtime cutover.
frontend/public/models/lagos-shadow-frontier.glb	Supplied 1.7 MB source model retained for the GDevelop path; not the active procedural Three.js map.
gdevelop/BlankLife/	Separate GDevelop project and exported project archive.
render.yaml; BUILD.md; docs/	Render service definition, build/deployment notes, architecture, product, setup and demo docs.

Build and run

`npm install` installs packages. `npm run dev` runs Vite and the Express API together. `npm run typecheck` validates TypeScript. `npm run build` type-checks then produces optimized frontend files in `dist/`. `npm start` starts the Node server, which serves the app and API. Render uses `npm ci && npm run build` followed by `npm start`.

Implemented prototype vs. not yet delivered

Working in the codebase	Not represented as a completed live service
Browser game/PWA, procedural 3D world, accounts, local SQLite persistence, available learning and career loops, file/video API routes, GitHub OAuth integration code, multiplayer/chat/Trust Protocol logic.	Native Android APK (the temporary APK wrapper was removed); Supabase runtime cutover and SQLite migration; guaranteed persistence on Render free tier; verified hundreds of concurrent users; live investment or payment processing; actual dispatch providers; phone/audio/video calls or end-to-end encrypted chat; live stadium broadcasts; unrestricted OS shell terminal; fully explorable interior geometry for every building.

Capacity note: Spatial rooms and client-side limits reduce work, but 300–700 concurrent players have not been load-tested or established. That target needs shared realtime state, horizontally scaled services, durable database/object storage, monitoring, and capacity testing.

Prepared from the repository documentation and source tree on 19 October 2026. Use README.md, ARCHITECTURE.md, BUILD.md, GAME_DEMO.md, and docs/README.md as the detailed living references.